TESTABLE WHITEBOX . TOOL VALIDATION AUTOMATION

Workflow Guide

How the dev branch script works, file by file, and how to run it step by step

BRANCH	LATEST COMMIT	WRITTEN ON	STACK
dev	707f5da	28 Oct 2026	Playwright, TypeScript, Node 22

What is inside

1 **The big picture**
what the script does in one minute

2 **The pipeline**
browser lane and background lane, side by side

3 **Main flow**
the 15 steps on Testable

4 **Background worker**
hand-off, S3 check, SharePoint, Teams

5 **How to use it**
one-time setup and every-run checklist

6 **File map**
which file does which job

7 **.env reference**
every setting, what to put, defaults

8 **Result folders**
what each output file is

9 **Troubleshooting**
symptom, cause, fix

10 **Cheat sheet**
commands and words in one page

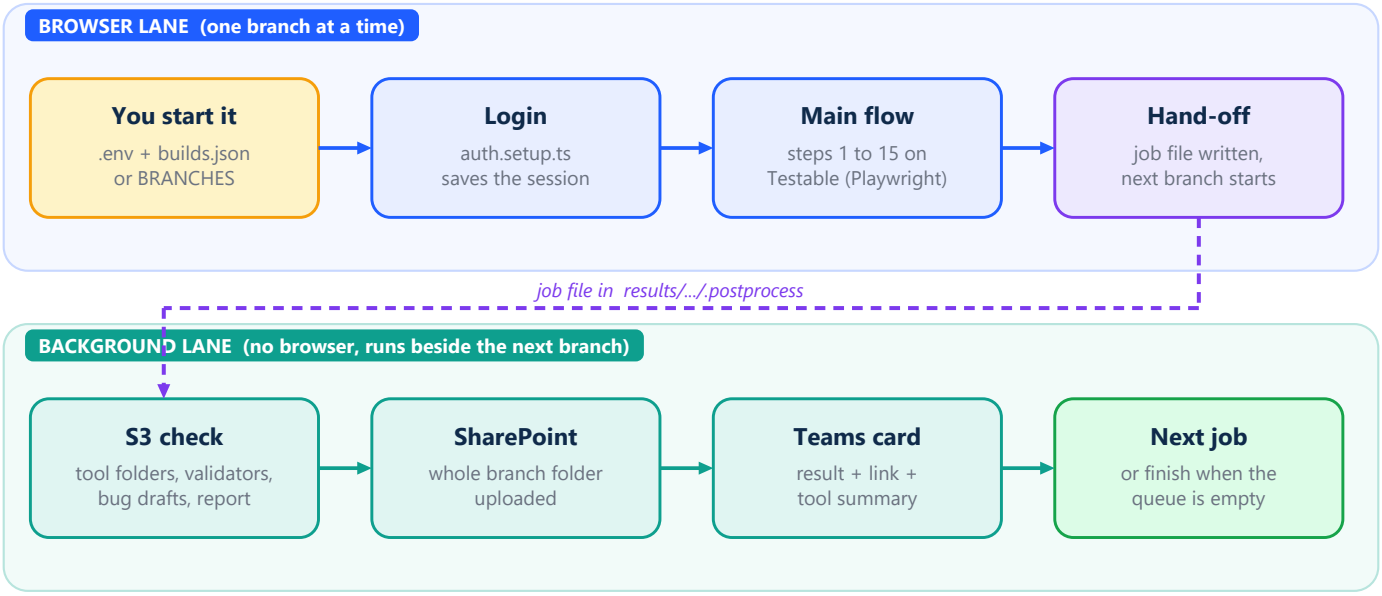
Who this is for
Anyone who runs the script, reads its results, or needs to change it. You do not need to know the code to follow sections 1 to 5, 7, 8, 9 and 10. Section 6 is for people who open the files.

1 The big picture

what the script does in one minute

The script tests a branch on the **Testable** platform exactly like a person would, then checks what the platform wrote to **AWS S3**, and sends the result to the team. You give it one or more **GitHub branch links**. For every branch it does this:

- **Runs the branch on Testable** with a real browser (Playwright): connects the repository, picks the branch, presses Execute, waits for the analysis to finish, and takes screenshots and the taxonomy report.
- **Checks the evidence in S3**: for every technique it opens the tool's folder, reads the files, and says **PASS**, **EMPTY** or **FAIL**. Language validators judge whether each tool behaved correctly for that language and version.
- **Writes bug drafts** (PDF, screenshot, Jira text) for the problems that belong to the platform. Nothing is sent to Jira automatically.
- **Uploads the whole branch folder to SharePoint** and **posts a card in Teams** with the result and a link.



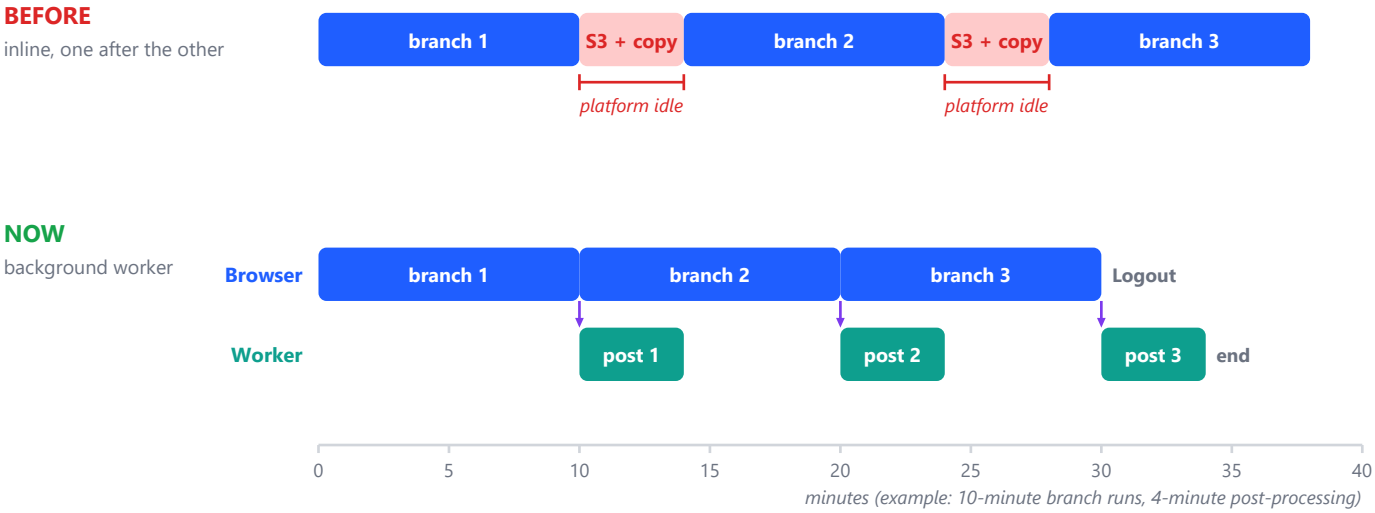
The key idea of this branch: after the browser part of a branch is finished, everything that does not need the browser (S3 check, SharePoint, Teams) moves to a **background worker**. The next branch starts on the platform right away, so the platform is never waiting for your reports to be written.

Three things to remember
1. The browser lane runs **one branch at a time**. 2. The background lane works on branch N **while** branch N+1 is executing. 3. Every setting lives in **.env** (section 7). A change to .env always wins over a value typed in the terminal.

2 The pipeline

why the platform no longer sits idle

Before, the script finished all the reports of a branch before it started the next one, so the platform waited. Now the reports are made by the worker beside the browser.



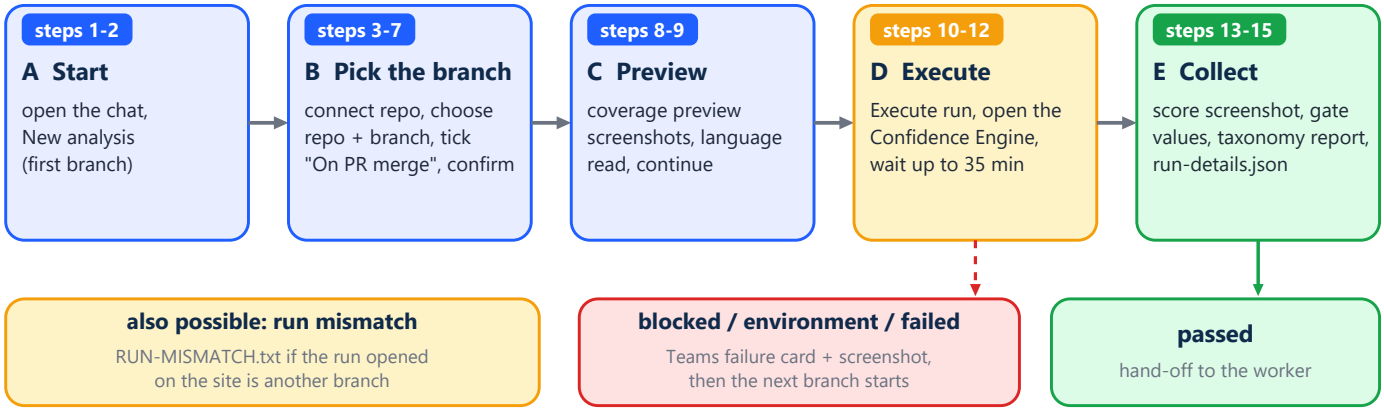
The grey-red gaps in the top lane are the time the platform sat idle. In the bottom lane the browser starts branch 2 straight after branch 1, and the worker handles branch 1 in the background. Only the **last** branch's post-processing happens after the last execution ends, and the final test waits for it. The minutes in the picture are examples; real runs take about 10 minutes per branch and about 3 to 5 minutes of post-processing.

The order of every run

#	Moment	What happens	Where you see it
1	Start	Branches are read from **BRANCHES** or **builds.json**. The run folder **results/merge-flow_<date>_<time>** is created.	terminal / Test Explorer
2	Login	**auth.setup.ts** logs in with the Testable account from .env and saves the session.	test "auth.setup.ts"
3	Branch N, browser	Main flow steps 1 to 15 (section 3).	test "Whitebox run - <branch>"
4	Hand-off	When branch N passes, a job file is written and the worker is started (if not already running). Branch N+1 starts.	line "handed to the background worker"
5	Branch N, worker	S3 check, bug drafts, SharePoint upload, Teams card (section 4). Runs beside branch N+1.	lines starting with **[post] [branch]**
6	Logout	After the last branch the browser logs out of Testable.	test "Logout"
7	Wait for the worker	The last branch's post-processing finishes; the worker ends.	test "Background post-processing finishes"

3 Main flow

the 15 steps the browser does for each branch



Which steps stop a branch
Steps **1, 3, 4, 5, 6, 7, 9, 10, 11 and 12** are *critical*: if one fails, this branch stops and the next branch starts. Steps 2, 8, 13, 14 and 15 are recorded but do not stop the branch on their own; a failure there still marks the branch failed at the end.

Step	What the script does	Function in pages/MergeFlowPage.ts	Saved as
0	Optional: with **DUMMY_PR=yes** it pushes a comment line to the branch's README so the platform sees a new commit.	utils/dummyCommit.ts	-
1	Opens the Chat Execution Assistant (later branches use the sidebar chat icon).	openChat	evidence/<branch>_1_PASS.png
2	First branch only: "Can I safely deploy this commit?" then New analysis / Refresh repos.	startNewAnalysis	-
3	Change repository / branch, then Connect or Import repository. Waits for the repo sync (cap 14 min).	connectRepo	-
4	Clicks the repository, then Configure repository.	chooseRepo	-
5	Selects the branch: "+N branches", search, tick, Done.	chooseBranch	-
6	Makes sure the "On PR merge" trigger is ticked.	ensurePrMergeTicked	evidence screenshot
7	Confirms the repository. The site may answer "Branch unavailable": the branch is then **blocked**.	confirmRepo	errors/site-error.txt (if blocked)
8	Coverage preview: repo structure, what will be analysed, uncovered areas. Reads Build type, Language and Language version.	coveragePreview, saveRepoDetails	01_ 02_ 03_ screenshots, repo-details.json
9	Continues from the preview / Review branches. Stops if the site blocks the analysis.	continueFromReview	-
10	Run plan, then Execute run. Retries if the run does not start.	executeRun	evidence screenshot
11	Execution status, then Open Confidence Engine.	openConfidenceEngine	-
12	Waits for the analysis to finish (**RUN_TIMEOUT_MIN**, default 35 minutes).	waitForRunToFinish	05_execution_status.png
13	Screenshot of the PR Confidence Analysis score.	screenshotPrScore	04_pr_confidence_score.png
14	Reads every technique x classification gate value from the page. With **GATE_SHOTS=on** it also takes a screenshot of each gate.	captureUiScore	ui-values.json, gates/
15	Downloads the taxonomy report, builds the report with the UI value column, saves the run record. Checks that the run on the site is the branch you asked for.	downloadTaxonomyReport, saveRunRecord, mergeTaxonomy()	taxonomy-gate-<run>.html, ...-with-ui-values.html, run-details.json

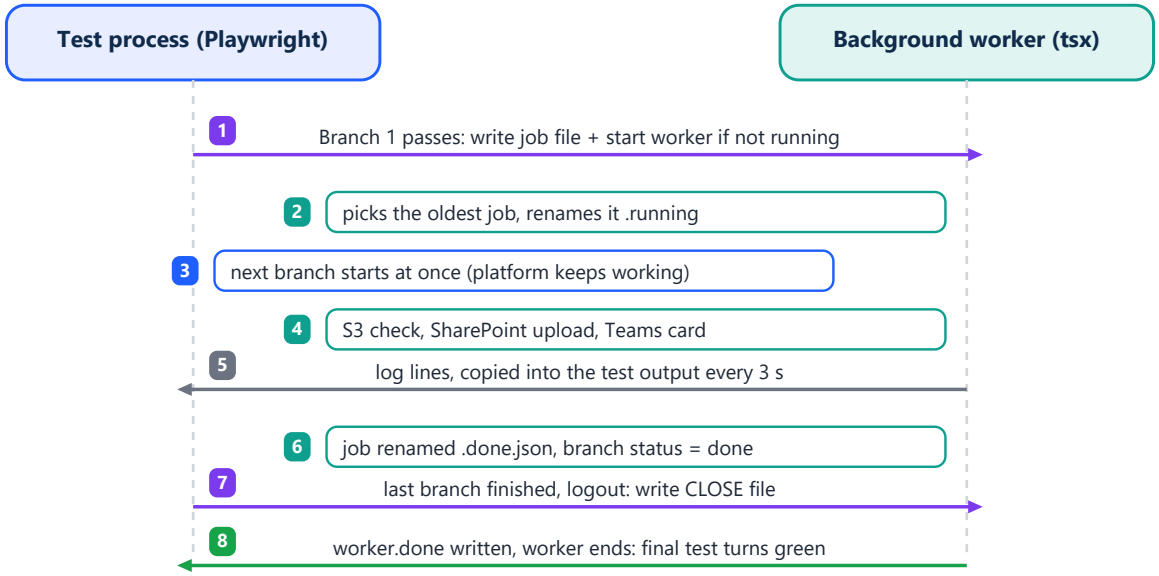
When a branch does not pass
A failed, blocked (site message) or environment problem (site down, repo sync stuck) ends that branch only. A screenshot and the network log are saved in **errors/**, a failure card goes to Teams straight away, and the next branch starts. A branch that failed has no background post-processing, because there is no run to check.

4 Background worker

what happens to a passed branch after the browser is done

4.1 The hand-off

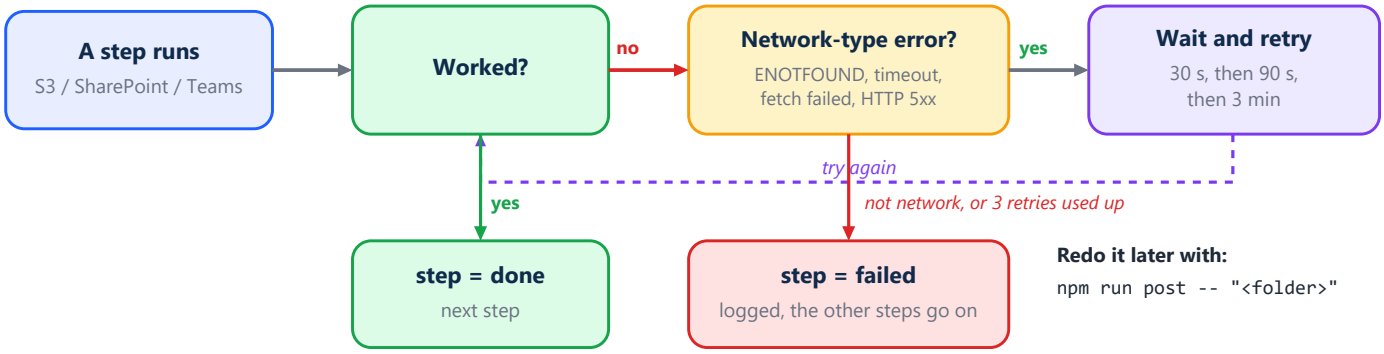
The browser test and the worker talk through plain files in **results/<run>/.postprocess/**, so nothing is lost if either side stops. This is the conversation for a run with several branches:



File in .postprocess/	Meaning
<time>-<branch>.job.json	a branch waiting to be post-processed
...job.json.running	the worker is working on it now (renamed back to .job.json if the worker dies, so the next worker redoes it)
...done.json / ...failed.json	finished jobs
worker.pid, CLOSE, worker.done	the worker's registration, "no more jobs are coming", and "I have finished"

4.2 What the worker does for each branch

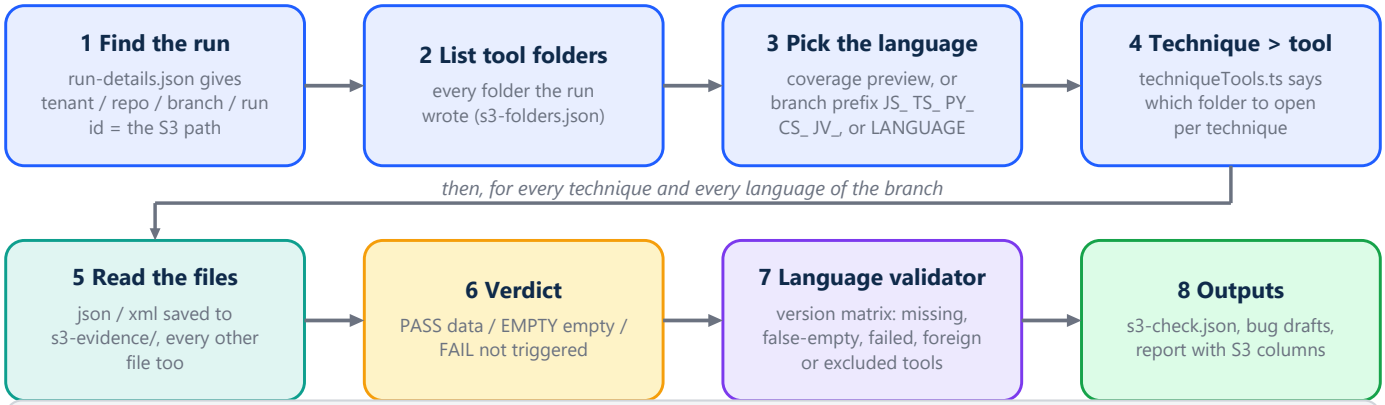
Order	Step	Details	If it fails
1	**S3 check**	Needs AWS keys in .env. Reads the run's tool folders, judges every tool, writes s3-check.json, bug drafts and the report with S3 columns (section 4.3).	logged; SharePoint and Teams still run
2	**SharePoint**	Uploads the whole branch folder to Automation Evidences Dump / date / branch (section 4.4).	logged; Teams still runs
3	**Teams**	Posts the success card with the evidence link and the tool summary (section 4.5).	marked failed, redo with npm run post



Network problems (DNS, timeouts, HTTP 5xx, "fetch failed") are retried three times after 30 seconds, 90 seconds and 3 minutes. Every branch has a status file **postprocess.json** with *queued, running, done* or *failed* for S3, SharePoint and Teams.

4.3 The S3 check, explained

Every tool the platform runs writes its files under one S3 path built from the run: **development/cell-001/<tenant>/<project>/<repository>/<branch id>/<branch>/<commit>/<run id>/<tool>/...**. The check follows eight steps:



The secondary tool is used only when the primary tool's folder is missing. A "primary N/A" row checks only its secondary tool.
When two languages share a folder (for example lizard), the file names inside (.js .ts .py .cs .java) say which language a file is for.

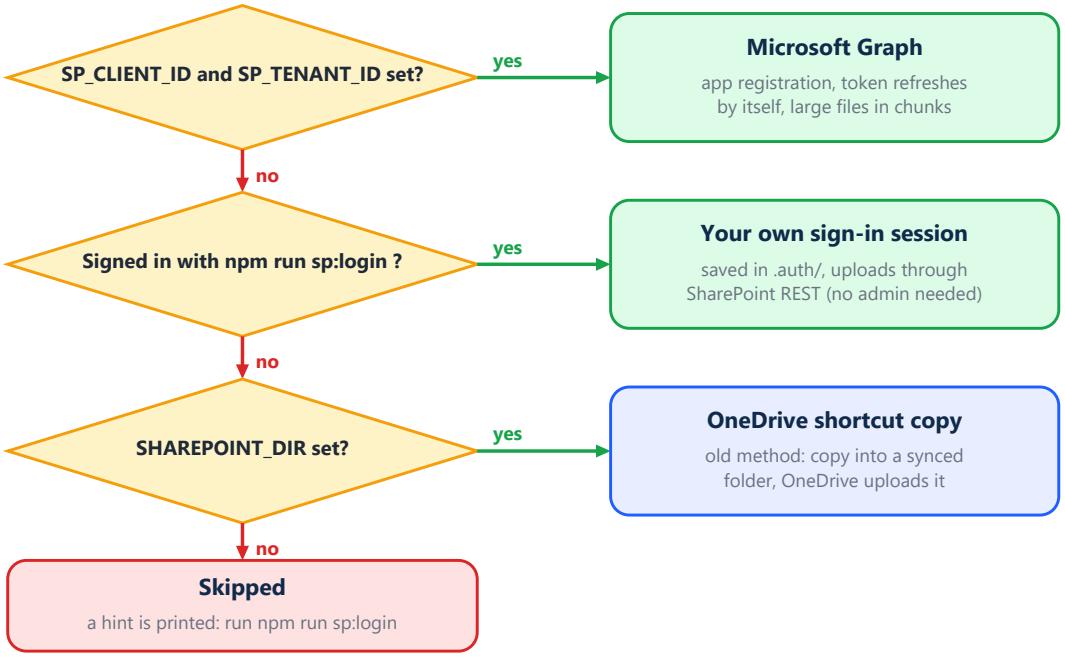
Verdict	Meaning	Typical reason
PASS	The tool was triggered and its file holds data.	e.g. lizard.xml with 123 items
EMPTY	Triggered, but the file is empty or the tool said "not applicable / skipped".	status=not_applicable, baseline_missing, empty list
FAIL	Not triggered: no folder, no json/xml file, or only a failure.json.	no folder jacoco in this run
N/A	The tool does not apply to this row.	primary tool not applicable for this classification

Language	Validator file	What it judges
Python	utils/pyValidate.ts	tool list per Python version, what each tool's file must hold
JavaScript	utils/jsValidate.ts	tool list per Node version, package manager and layout
TypeScript	utils/tsValidate.ts	same as JavaScript, with the TypeScript tool names
C#	utils/csValidate.ts	tool list per .NET version and target
Java	utils/javaValidate.ts	tool list per JDK version and build tool

The validators produce **defect tags** for platform problems: *[Planning Defect]* (a tool that should run is missing, or one that must be excluded ran), *[False Empty Defect]* (empty report after scanning 0 files), *[Runner Execution Defect]* (the tool crashed) and *[Language Contamination]* (a tool of another language was planned). Failures caused by the customer's own repository are listed as repo-side and are **not** filed as platform bugs. For each platform defect **bugReport.ts** writes a 2-page PDF, a screenshot of the report row and a Jira text file into **bugs/**.

4.4 SharePoint

The upload target is the team folder **Automation Evidences Dump** on the Testable-ExecutionTeam site. The script creates a folder for the day (**DD-MM-YYYY**) and one folder per branch inside it. A second run of the same branch name on the same day goes to **<branch>_RUN2**, then **_RUN3**. Three upload methods exist; the script picks the first one that is set up:



Where it lands

Automation Evidences Dump / DD-MM-YYYY / <branch>

Same name again the same day: <branch>_RUN2, _RUN3 ...

Sign in once

Each developer runs **npm run sp:login** one time. A browser opens; sign in with your Microsoft work account and approve the prompt. The session is saved in .auth/ and reused. If your company's policy ends the session, the upload says "sign-in expired" and you run the command again. Files larger than 240 MB are not uploaded and are listed in sharepoint-copy-errors.txt.

4.5 Teams

Card	When	Contents
Success	sent by the worker, after S3 and SharePoint	Branch name, repo, environment (QA/DEV), minutes; buttons "Open the run" and "Open evidence folder"; per language: tools expected and OK, tools with a bug (draft reports saved) or "No tool bugs found".
Failure	sent by the browser test at once	Branch name, status (failed / blocked by the site / environment problem), the reason, and a small screenshot.
Did not start	sent by the queue server	A round stopped before a branch could begin (for example the site is down at login).

5 How to use it

one-time setup, then the same checklist every run

5.1 One-time setup (per PC)

#	Do this	Command or place
1	Install Node.js 22 and Git. Clone the repository and switch to the dev branch.	git checkout dev
2	Install the packages (this also installs tsx, which runs the background worker).	`npm install`
3	Install the Playwright browser if it is missing.	`npx playwright install chromium`
4	Copy **.env.example** to **.env** and fill it (section 7). At least: Testable login, AWS keys, optionally the Teams webhook.	.env in the project folder
5	Sign in to SharePoint once.	`npm run sp:login`
6	Make sure the PC will not sleep during a run. The script keeps Windows awake by itself while it runs.	Windows power settings

5.2 Every run

#	Do this	Why
1	Refresh the AWS keys in .env (AWS access portal, QA-S3-Readonly, Access keys). They last only hours to about a day.	otherwise the S3 step fails with ExpiredToken
2	Put your branches in **builds.json** or set **BRANCHES** (see below). One branch link per entry.	the list of branches to test
3	Start the run (terminal or Test Explorer, below). A Chromium window opens. Do not close it or lock the PC.	the browser lane needs it
4	Do not edit .env or a playwright.config file while it runs.	the VS Code extension restarts its server and stops the run
5	Watch the **[post] [branch]** lines. When the last test "Background post-processing finishes" turns green, everything is done.	live progress of the worker
6	Check Teams, SharePoint and the results folder (section 8).	the deliverables
7	Afterwards run **git checkout builds.json** so your test list is not committed.	keeps the repo clean

5.3 Two ways to start

A. Terminal (PowerShell)

```
$env:BRANCHES="https://github.com/OWNER/REPO/tree/BRANCH1,https://github.com/OWNER/REPO/tree/BRANCH2"
npm test
```

Several branches: separate the links with commas. Several repos in one run is fine, the link carries the repo.

B. Test Explorer (Antigravity / VS Code)

1. Open **builds.json** and write `{ "branches": ["link1", "link2"] }`
2. Press the refresh icon in the Testing panel.
3. Turn on **Show browser**.
4. Select the **flow** project and run the whole project, not one single test.

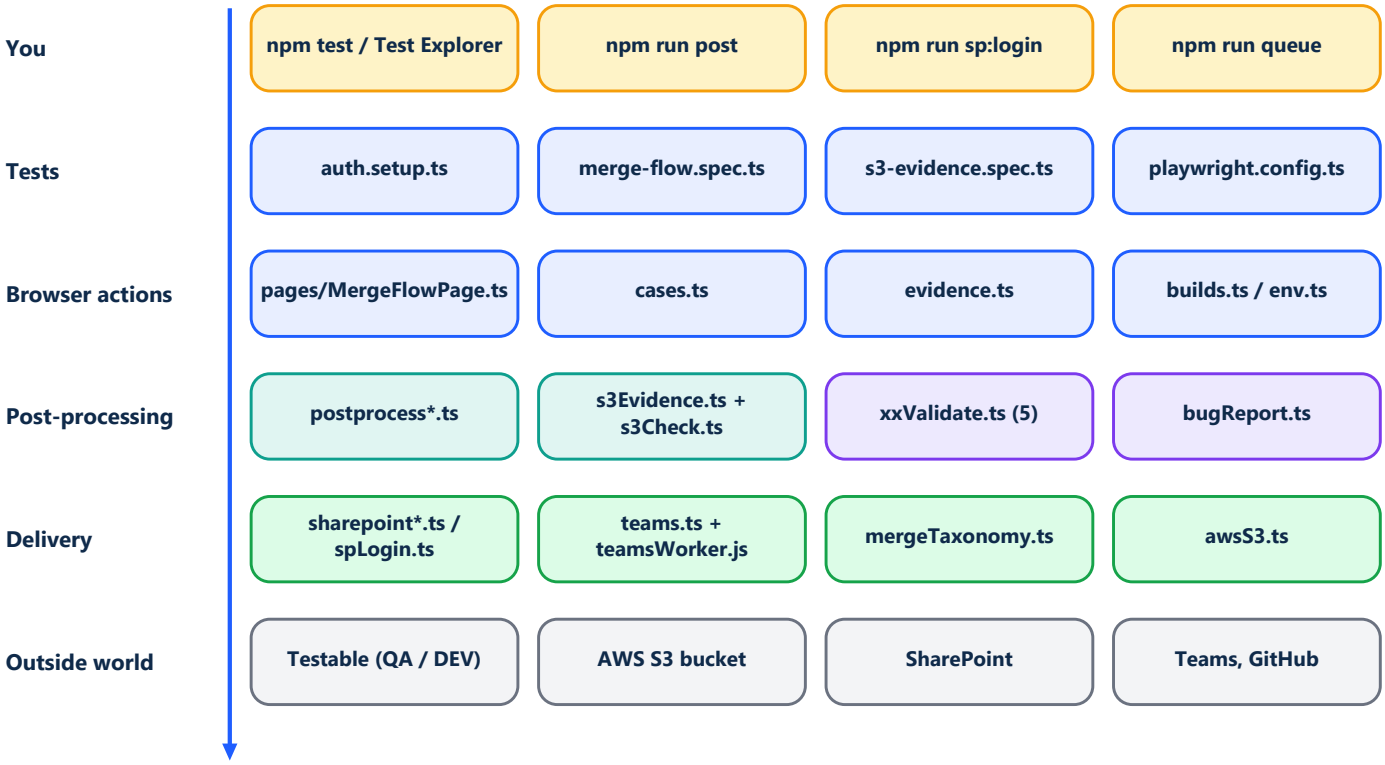
5.4 Commands

Command	What it does
`npm test`	The full run: login, every branch, logout, wait for the worker.
`npm run setup`	Only the Testable login (saves .auth/user.json). Useful to test your credentials.
`npm run sp:login`	One-time SharePoint sign-in for you.
`npm run post -- "<branch folder>" --repo owner/name`	Repeats S3 check, bug drafts, SharePoint and Teams for one finished branch. Use it after a network problem or expired keys.

Command	What it does
`npm run s3`	Old way: re-checks S3 for a results folder (S3_RESULTS_DIR, S3_FORCE=1). Normally not needed.
`npm run queue`	Starts the optional branch queue server (section 10).

6 File map

which file does which job



Each layer calls the one below it. Read the picture top to bottom to see how a command you type ends up as a Teams card. The table below lists every file.

File	What it does
Project root	
package.json / package-lock.json	npm scripts (test, setup, s3, queue, sp:login, post) and packages (Playwright, AWS SDK, exceljs, tsx, dotenv).
playwright.config.ts	Three projects: setup (login), flow (the branch tests, needs setup) and s3 (old re-check). One worker, no retries, a failed branch never stops the run, headed browser, timeouts.
tsconfig.json	TypeScript settings (strict).
builds.json	The branch list. The queue server rewrites it; keep it empty in git.
.env / .env.example	All settings (section 7). .env is private and git-ignored; .env.example is the template.
.gitignore	Keeps .env, .auth/, results/, queue data and logs out of git.
testable-merge-flow-HANDOFF.md	Older hand-over notes. Parts are out of date (for example Java and the gate screenshots default); this guide replaces it.
tests/	
auth.setup.ts	Logs in to Testable (normal email + password, or Okta SSO with LOGIN_MODE=okta) and saves the session in .auth/.
merge-flow.spec.ts	The main flow: one test per branch (steps 1 to 15), then Logout, then the test that waits for the worker.

File	What it does
s3-evidence.spec.ts	Stand-alone S3 re-check of an old results folder (npm run s3).
pages/	
MergeFlowPage.ts	Every action on the Testable site: opening the chat, connecting the repo, choosing the branch, Execute, waiting, reading the gates, downloading the report, logging out. Detects "blocked" and "environment" problems.
utils/ - running the browser flow	
env.ts	Loads .env, picks QA or DEV, builds the site URL, S3 bucket name and the saved-session path.
builds.ts	Reads the branches from BRANCHES, REPO or builds.json (accepts full GitHub /tree/ links).
cases.ts	Runs each step as a named test step, records pass/fail, and stops the branch only on critical steps.
evidence.ts	The run folder, step screenshots, summary.json, and the network log of failed requests.
dummyCommit.ts	Optional README comment commit so the platform analyses a fresh commit (DUMMY_PR).
utils/ - S3 check and validators	
awsS3.ts	Reads the S3 bucket with the AWS SDK (no browser).
s3Evidence.ts	The S3 check for one branch (the eight steps in section 4.3).
s3Check.ts	Turns a tool's json/xml into PASS / EMPTY / FAIL with a plain-English note.
techniqueTools.ts	The map: technique > primary tool > secondary tool, per language (JS, TS, C#, Python, Java).
pyValidate.ts, jsValidate.ts, tsValidate.ts, csValidate.ts, javaValidate.ts	The per-language validators and their version matrices; they produce the defect tags.
mergeTaxonomy.ts	Builds taxonomy-...-with-ui-values.html: adds the UI value column and the S3 Tool / JSON and S3 comment columns, and a header box with the S3 path and the validation summaries.
bugReport.ts	Bug drafts per platform defect: PDF, report-row screenshot, Jira text, bugs.json.
utils/ - background worker	
postprocess.ts	The three worker steps (S3, SharePoint, Teams) with retries and the postprocess.json status.
postprocessQueue.ts	Hand-off from the test: writes job files, starts the worker, copies its log into the test output every 3 s, waits for it at the end.
postprocessWorker.ts	The worker process: takes jobs one at a time, prefixes each log line with the branch name.
postprocessCli.ts	npm run post: run the post-processing of one finished branch by hand.
utils/ - delivery	
sharepointSession.ts	SharePoint upload with your own sign-in (REST). Sign-in itself: spLogin.ts.
sharepointGraph.ts	SharePoint upload through Microsoft Graph (needs an app registration); also holds the URL parser and the day-folder naming.
sharepoint.ts	Old OneDrive-shortcut copy (SHAREPOINT_DIR).
spLogin.ts	npm run sp:login.
teams.ts	Builds and posts the Teams cards (with retries). teamsWorker.js sends a failure card with a shrunk screenshot in a separate process so it can never hold up the run.
awsConsole.ts	Old AWS-portal login helper. Not used by the current flow.
queue-server/ (optional)	
server.js	The branch queue and its web page, fair rounds, stuck-run watchdog with auto-recovery.
mirror.js, freezeTracker.js, keepAwake.js	Copies log lines to /output, records why a run froze (PC sleep, killed process), keeps Windows awake.
public/index.html, public/output.html	The queue page and the live-output page.

File	What it does
Created while you work (not in git)	
.auth/	Saved sessions: Testable (user.json), SharePoint (sharepoint-session.json, sharepoint-token.json).
results/	Everything a run produces (section 8).
queue-server/data/	Queue state, logs, heartbeat and freeze-tracker files.

7 .env reference

every setting the scripts read

Create the file **.env** in the project folder (copy .env.example). One setting per line, **NAME=value**, no spaces around the equals sign. **The values in .env always win**: a variable typed in the terminal is ignored if the same name is in .env. (BRANCHES and REPO are the usual exceptions: keep them out of .env.)

Must have

Variable	What to put	Default / notes
TEST_ENV	**qa** or **dev**: which Testable site and S3 bucket to use.	qa
TESTABLE_EMAIL TESTABLE_PASSWORD	The Testable account for QA.	required for qa
TESTABLE_EMAIL_DEV TESTABLE_PASSWORD_DEV	The Testable account for DEV.	required when TEST_ENV=dev
AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN	Temporary keys of the QA-S3-Readonly role (AWS access portal, Access keys). Needed for the S3 check.	expire after hours to about a day; without them the S3 step is skipped

Delivery (optional but recommended)

Variable	What to put	Default / notes
TEAMS_WEBHOOK_URL	The Workflows webhook of the Teams channel ("Send webhook alerts to a channel"). Keep it secret.	empty = no Teams messages
SP_URL	Address-bar URL of another SharePoint parent folder.	Automation Evidences Dump (built in)
SP_TENANT_ID SP_CLIENT_ID	From the Entra app registration. Only for the Graph method; the sign-in session method needs neither.	empty = session method
SHAREPOINT_DIR SHAREPOINT_URL	Old OneDrive-shortcut method: the local folder, and the folder's address-bar URL for the Teams link. Change them every day.	not needed with sp:login
GITHUB_TOKEN	Lets bug reports read the branch's trigger file from private repos.	optional

Run behaviour (optional)

Variable	What it does	Default
POSTPROCESS	**background**: worker beside the next branch. **inline**: everything in the test, one branch after the other.	background
GATE_SHOTS	**on** takes a screenshot of every gate. Off still reads all gate values.	off
DUMMY_PR	**yes**: push a README comment to every branch first so a new commit is analysed. Needs git signed in with write access.	no
RUN_TIMEOUT_MIN	How long to wait for one analysis to finish.	35
REPO_SYNC_CAP_MIN	How long the repository sync may take before the branch is marked as an environment problem.	14
LANGUAGE	Force the language(s) for the S3 check, e.g. javascript,typescript. Normally detected.	detected

Variable	What it does	Default
LOGIN_MODE	**normal** (email + password) or **okta** (Sign in with SSO). OKTA_EMAIL / OKTA_PASSWORD, if set, are used for the Okta route.	normal
HEADLESS, PAGE_SCALE	HEADLESS=1 hides the browser. PAGE_SCALE (e.g. 0.9) shows more of the page in screenshots.	visible, 1
BUG_REPORTS	**off** turns bug drafts off.	on
KEEP_AWAKE	**off** stops the script from keeping Windows awake.	on
PY_IGNORE_TOOLS, JS_..., TS_..., CS_..., JAVA_IGNORE_TOOLS	Comma-separated tools the validator should not expect.	none
S3_BUCKET, AWS_REGION, BASE_URL	Override the bucket (qa-bucket-us2 / dev-bucket-us2), region (us-east-2) or site address.	follow TEST_ENV
AWS_PROFILE	Use an AWS SSO profile instead of the three keys.	none

Branch input and advanced

Variable	What it does
BRANCHES, REPO, BUILDS_FILE	The branch list: BRANCHES="link1,link2" (full GitHub /tree/ links), or REPO plus branch names, or another JSON file. Set them in the terminal, not in .env.
S3_RESULTS_DIR, S3_FORCE	For npm run s3: which results folder to re-check, and 1 to redo a branch that was already checked.
RUN_DIR	The output folder of the run. Set by the config; the queue server sets it for its rounds.
SP_DEBUG=1	Prints every SharePoint call (for finding problems).
QUEUE_PORT, QUEUE_CODE, QUEUE_ROUND_MAX, QUEUE_CMD, QUEUE_RECOVER_START_MIN, QUEUE_RECOVER_MAX_MIN, QUEUE_STUCK_MIN	Queue server: port (3000), access code for the page, most people per round (10), the command a round runs (npm test), and the watchdog times (6, 50 and 8 minutes).

A safe starting .env (replace the placeholders, never paste real secrets into a document)

```
TEST_ENV=qa
TESTABLE_EMAIL=your.name@company.com
TESTABLE_PASSWORD=your-password

AWS_ACCESS_KEY_ID=ASIA...
AWS_SECRET_ACCESS_KEY=...
AWS_SESSION_TOKEN=...
AWS_REGION=us-east-2

TEAMS_WEBHOOK_URL=https://...
POSTPROCESS=background
GATE_SHOTS=off
DUMMY_PR=no
```

8 Result folders

what every output file is

```
results/merge-flow_2026-10-28_1401/          one folder per run
  summary.json                               status and minutes of every branch
  postprocess.log                            the background worker's full log
  evidence/, evidence.jsonl                  step screenshots and the pass/fail record
  .postprocess/                              job files of the worker (section 4.1)
  <branch name>/                             one folder per branch
    01_repo_structure.png                    coverage preview screenshots
    02_what_we_will_analyze.png
    03_uncovered_areas.png
    04_pr_confidence_score.png               the PR score
    05_execution_status.png                  the execution status tick
    gates/                                   gate screenshots (only with GATE_SHOTS=on)
    ui-values.json                           every gate value read from the page
    repo-details.json                        build type, language, language version
    run-details.json                         run id, branch id, commit (the S3 path)
    taxonomy-gate-<run>.html                 the report as downloaded
    taxonomy-gate-...-with-ui-values.html    the finished report (UI + S3 columns)
    s3-folders.json, s3-files.json           what the run wrote to S3, file excerpts
    s3-evidence/                             a copy of every file of every tool
    s3-check.json                            every verdict, validator summaries, defects
    bugs/                                    Defect_Report_*.pdf, JIRA_*.jira.md, bugs.json
    postprocess.json                         status of S3, SharePoint and Teams
    errors/                                  failed branch only: failure.png, network.json,
                                               site-error.txt, teams-notify.log
    RUN-MISMATCH.txt                         only if the run on the site was another branch
    sharepoint-copy-errors.txt               only if some files could not be uploaded
```

The same branch in SharePoint

General / QA-Testable / Testable Whitebox Tool Execution / Automation Evidences Dump / 28-10-2026 / <branch name> / followed by the whole branch folder above, including s3-evidence and bugs.

How to read the finished report

Part of the report	Meaning
Box at the top	The S3 path of the run, a link to the AWS console, and one line per language: tools expected, how many OK, clean, empty, false-empty, failed, missing (Planning Defect), tools that ran when they should not have, and foreign-language tools.
UI value column	The value the Confidence Engine page shows. Same as the report: plain. Different: red, with the report's value in the tooltip. Not found on the page: a grey dash.
Tool / JSON and S3 comment columns	The tool's folder and files in S3 (a saved copy and a link to the AWS console), the PASS / EMPTY / FAIL tag, and the note. A "secondary" tag means the primary tool's folder was missing and the secondary tool was used.

9 Troubleshooting

symptom, likely cause, what to do

What you see	Likely cause	What to do
All tests skipped, "No branches configured"	builds.json is empty and BRANCHES is not set	Add the branches (section 5.3) and refresh the test list
Login fails in auth.setup.ts	Wrong TESTABLE_EMAIL / PASSWORD, or the site expects Okta	Run npm run setup; check .env; try LOGIN_MODE=okta
Branch BLOCKED: "Branch unavailable"	The site's repository list is out of date (the branch exists on GitHub)	Run that branch again later; check the name on GitHub
ENVIRONMENT: repo sync did not finish	The Testable QA backend is slow or down	Try again later; raise REPO_SYNC_CAP_MIN if needed
S3 step: ExpiredToken or "AWS keys expired"	The temporary AWS keys ran out	Paste fresh keys in .env, then npm run post -- "<branch folder>"
S3 or SharePoint or Teams: ENOTFOUND, fetch failed	Short network or DNS outage	Automatic retries after 30 s, 90 s, 3 min; if it still failed use npm run post
SharePoint: "sign-in expired" or "not signed in"	The saved session ended or was never made	Run npm run sp:login
SharePoint: some files failed	File over 240 MB, or a very long path	See sharepoint-copy-errors.txt in the branch folder
No Teams message for a passed branch	Webhook empty or wrong, or the network was down	Check postprocess.json (teams: failed?), fix .env, npm run post
No [post] lines after a branch passed	POSTPROCESS=inline in .env, or npm install was not run (tsx missing)	Set POSTPROCESS=background; run npm install
The run stopped by itself while the PC was in use	Editing .env or playwright.config.ts restarts the VS Code Playwright server; or the PC slept	Do not touch those files during a run; keep the PC awake
Browser is not visible in the Test Explorer	"Show browser" is off	Turn it on in the Playwright section
RUN-MISMATCH.txt in a branch folder	The run opened on the site belongs to another branch	Do not trust that folder; run the branch again
A terminal value is ignored	.env wins over the terminal	Change the value in .env

First place to look
For any problem after a branch finished: open **<branch folder>/postprocess.json**. It tells you which of S3, SharePoint and Teams worked and shows the error text of the one that did not. The full story is in **postprocess.log** in the run folder.

10 Cheat sheet

commands, words and the optional queue

Words used in this guide

Word	Meaning
Branch folder	results/<run>/<branch name>/ : everything for one branch.
Main flow / browser lane	The 15 steps done with Playwright on the Testable site.
Worker / background lane	The separate process that does S3, SharePoint and Teams without a browser.
Hand-off	The moment a passed branch is given to the worker (a job file is written).
Gate / technique / classification	The rows of the taxonomy report: a technique (for example Cyclomatic Complexity) has classifications, each with a value.
Primary / secondary tool	The tool expected for a technique, and its back-up when the primary tool's folder is missing.
Platform defect	A problem in the Testable platform (tool not planned, empty report, crash, wrong language) as opposed to a problem in the customer's repo.
Blocked / environment / failed	The three ways a branch can end without passing: the site refused, the site or network was unavailable, or a step failed.

One-minute routine

Before	During	After
1. Fresh AWS keys in .env 2. Branches in builds.json or BRANCHES 3. npm run sp:login (once)	4. npm test, or run the flow project 5. Do not touch .env or the config 6. Watch [post] [branch] lines	7. Teams card + SharePoint folder 8. postprocess.json if something is missing 9. git checkout builds.json

The optional branch queue

When many people add branches, run **npm run queue** and open **http://localhost:3000** (the team address is printed). Everybody adds their GitHub branch links with their name. The owner types **start** in the terminal. A round takes one branch from each waiting person, the person served longest ago first, and runs it with the normal flow. Set **QUEUE_CODE** in .env before sharing the page outside the office network. A run that freezes at the start of a branch is stopped and restarted by itself.



The owner types start | pause | stop | clear | list | remove <id> | status in the terminal that runs the server.

About this document
It describes the dev branch at commit 707f5da. If you change a file in the file map, update the matching section here. The script prints what it does; when in doubt, trust the log lines and postprocess.json over this guide.